

Universidade do Minho

Visualização de Dados e Informação

Mestrado em Engenharia e Ciência de Dados

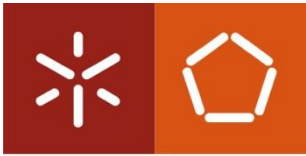
2025/2026

Telmo Adão

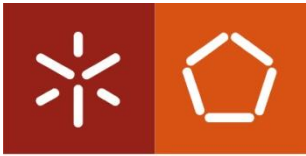
Departamento de Sistemas de Informação

Universidade do Minho

Este conjunto de slides contém algum material didático disponibilizado pelo Professor Pedro Branco

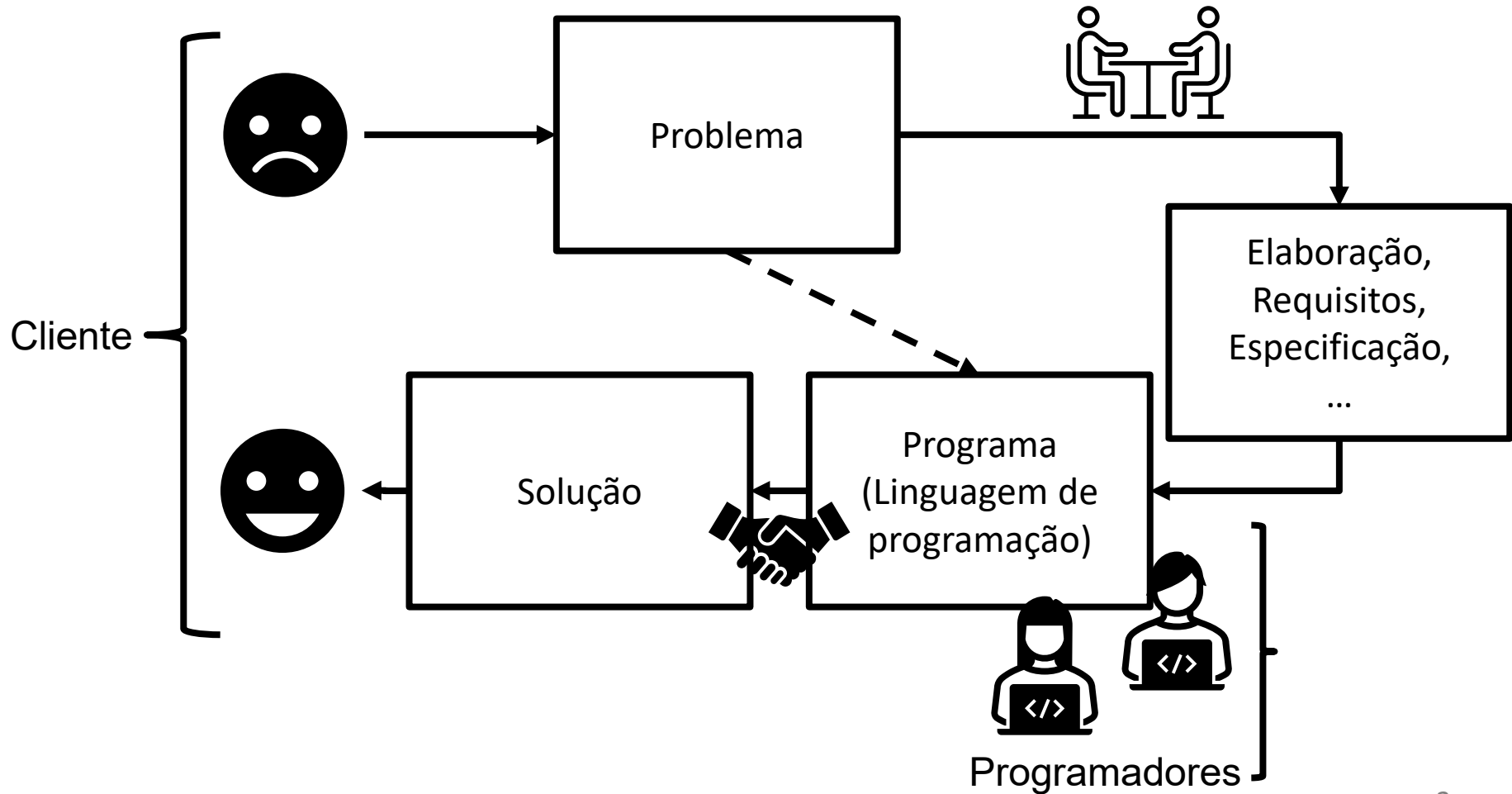


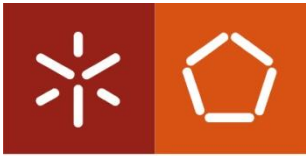
- Revisões de Python
 - Variáveis
 - Operadores
 - Operações de I/O
 - Estruturas de decisão
 - Estruturas de repetição
 - Listas, expressões “comprehension”, tuplos e dicionários
 - Strings
 - Funções
 - Algumas funções interessantes da biblioteca math
 - Ficheiros
 - ... e alguma visualização



Universidade do Minho

Engenharia de software

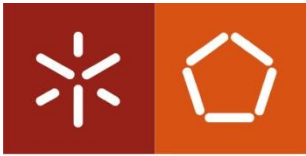




Universidade do Minho

Algoritmo

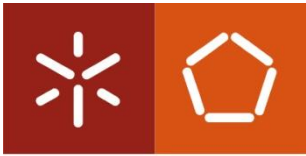
- Surge da necessidade de resolução de um problema
- Consiste na descrição de um conjunto de passos
- É independente da linguagem de programação



Algoritmo – Exemplos

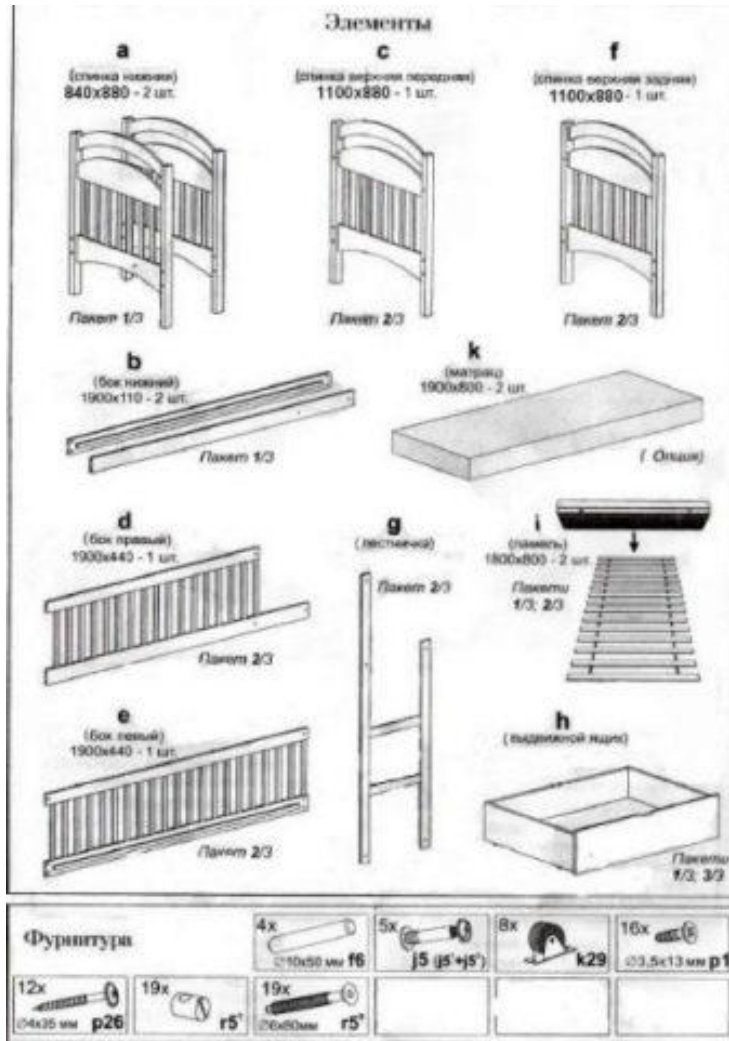
- Receita leite creme

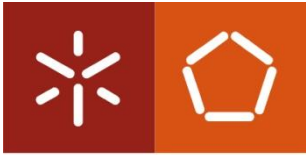
1. Preparar ingredientes: 1 l de leite meio-gordo, 7 gemas de ovo M, 150 g açúcar + qb para queimar, 2 c. sopa farinha Maizena qb, casca de limão.
2. Colocar as gemas numa taça e misturar com uma vara de arames e lentamente adicione a farinha
3. Colocar de seguida o açúcar e voltar a misturar, e por fim o leite frio
4. Levar a mistura num tacho ao lume e juntar a casca de limão e um pau de canela previamente lavados
5. Deixe engrossar em lume brando
6. Quando engrossar, desligar o lume e reserve
7. Verter a mistura numa taça grande ou distribua por taças individuais e polvilhar com açúcar queimado com um ferro próprio ou um maçarico de cozinha



Universidade do Minho

Algoritmo – Exemplos

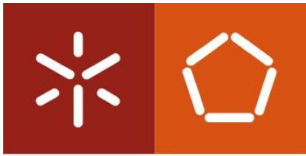




Universidade do Minho

Programação

- É a “digitalização” do algoritmo e resulta, normalmente, num artefacto de software que faz interface com um utilizador.
- Programa: é uma coleção de instruções que descrevem uma tarefa a ser realizada por um computador.

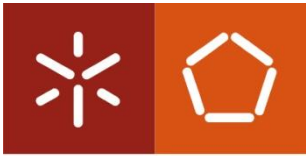


- Porquê Python

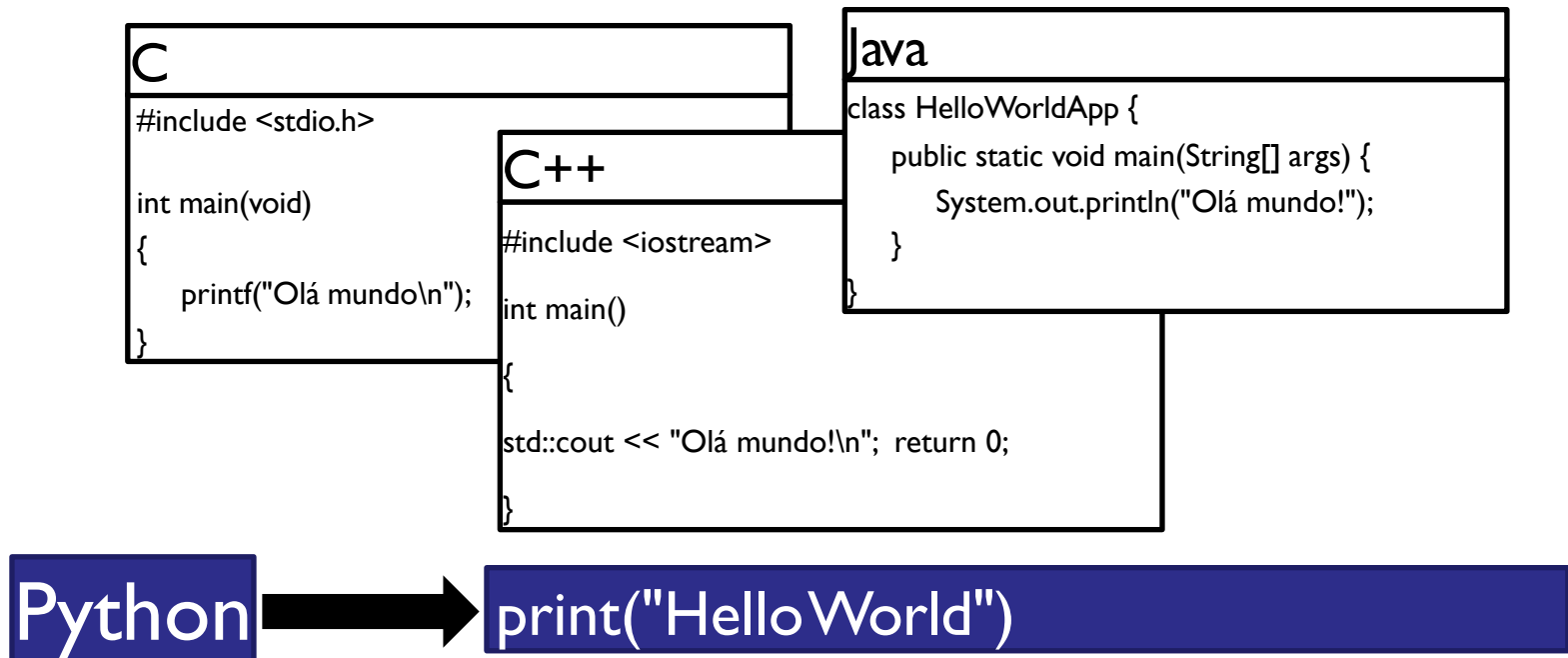
...das linguagens mais usadas atualmente!

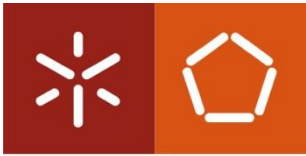
Language Rank	Types	Spectrum Ranking
1. Python	  	100.0
2. C++	  	99.7
3. Java	  	97.5
4. C	  	96.7
5. C#	  	89.4
6. PHP		84.9
7. R		82.9
8. JavaScript	 	82.6
9. Go	 	76.4
10. Assembly		74.1

IEEE spectrum 2018



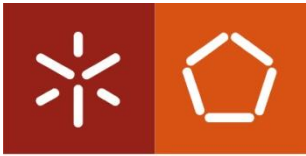
- Leve do ponto de vista sintático, comparativamente com outras linguagens:





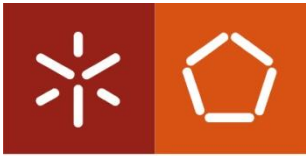
Python → variáveis

- Guardam dados em memória
- Podem mudar ao longo do programa (daí o nome)
 - Exemplo: dados introduzidos pelo utilizador, resultados intermédios de cálculos, dados que serão processados pelo programa...
- Tudo o que quiserem que o computador se "lembre" tem de estar guardado numa variável



Python → variáveis

- Usa-se o símbolo “=” para atribuir um valor à variável: `a = 5`
- Uma variável pode ter qualquer valor.
- Ao contrário de muitas outras linguagens, o Python atribui o tipo “on-the-fly”.



Universidade do Minho

Python → variáveis

Thonny - /Users/pbranco/Downloads/Untitled.py @ 5 : 15

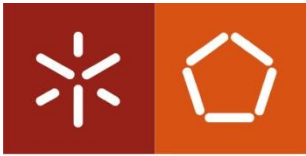
```
Untitled.py x
minha_variavel = 10
meuNome = "pedro"
v_ou_f = True
```

Variables

Name	Value
meuNome	'pedro'
minha_variavel	10
v_ou_f	True

Shell

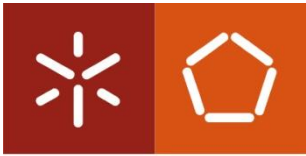
```
>>> %Run Untitled.py
>>>
```



Universidade do Minho

Python → variáveis

- Tipos
 - Numéricas
 - int, float
 - Exemplo: 1, -4, 1e45, 0.123456...
 - Booleanas: tipo boolean → True / False (apenas este 2 valores)
 - Strings: sequências de caracteres delimitadas por " ou '
 - Exemplo: "Isto é uma string 123"
 - Listas, tuplos, dicionários,
 - Instâncias de objetos, etc.



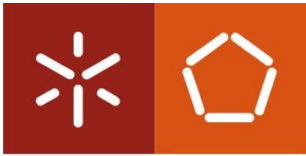
Python → variáveis

```
a = 5           # variável do tipo inteiro
b = -10         # inteiro
pi = 3.1415     # float

n = "nome"      # variável do tipo string
a = "100"       # string

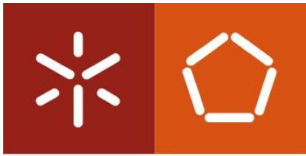
b = True        # variável do tipo boolean

l = [1,3,8,13]  # variável do tipo lista
```



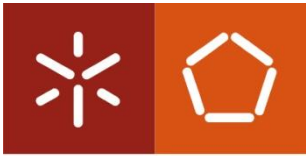
Python → variáveis

- Nomes das variáveis:
 - Podem conter letras e números e podem usar o símbolo “_”;
 - Não usar assentos, cedilhas ou outros caracteres especiais!
 - O nome deve informar o que a variável representa (clareza, precisão);
 - Há diferença entre maiúsculas e minúsculas (*case-sensitive*)
 - ex. variável cont não é o mesmo que variável Cont
 - Por convenção usam-se letras minúsculas nos nomes.



Python → operadores

- Fazem, cálculos com números ou variáveis, por exemplo operadores aritméticos:
 - Soma → +
 - Subtração → -
 - Divisão → /
 - Multiplicação → *
 - Potencia → **
 - Divisão Inteira → //
 - Módulo → % (resto divisão)



Python → operadores

exemplo com operadores

```
desconto = 0.1
```

```
preco = 100
```

```
valor_desconto = desconto * preço
```

```
precoApagar = preco - valor_desconto
```

```
q = 3**2
```

3 ao quadrado

```
d = 5 / 2
```

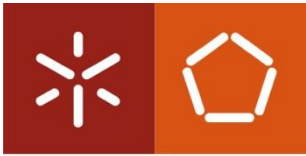
2.5, d é do tipo float

```
d = 5 // 2
```

#2 (divisão inteira), d é do tipo int

```
r = 5 % 2
```

#1 (resto da divisão inteira)

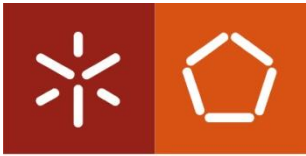


- **Saída:** exibir dados (ecrã, disco, rede, ...)

```
print( 3.1415)
print ("olá")
a = -5                                #representa "saída"???)
print ( a )
b = True                              #representa "saída"???)
print( b)
nome = "Camões"                       #representa "saída"???)
print ("o meu nome é", nome)
```

- **Entrada:** ler dados do utilizador (teclado, disco, rede...)

```
nome = input("como te chamas ?")
p = input("preço ?")                #Cuidado... Isto não vai ser lido como número! Vamos ver depois...
```



- **Entrada de dados**

- **IMPORTANTE** → a variável que guarda os dados introduzido (input) é sempre do tipo *string*, mesmo que o intuito seja a leitura de um número

- A Shell não mente:

```
p = input("preço ?")
```

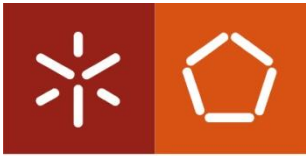
```
>>>
```

```
preço ? 20
```

```
type(p)
```

```
>>>
```

```
<class 'str'>
```



Python → I/O

- Entrada de dados: como ler o devidamente uma variável numérica?

... é necessário converter a string para int ou float, com os comandos int() e float()...

#assim...

```
p = input("preço ?")
```

```
preco = int(p)
```

converte p para inteiro

```
d = input("desconto ?")
```

```
desconto = float(d)
```

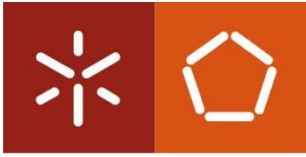
converte d para float

```
valor_desconto = desconto * preco
```

#agora podemos calcular...

```
precoApagar = preco - valor_desconto
```

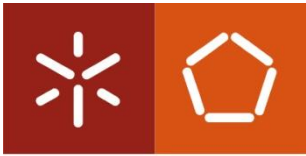
```
print(precoApagar)
```



Universidade do Minho

Python → estruturas de controlo

- **Instruções que permitem repetir instruções ou executar condicionalmente instruções**
 - Estruturas de decisão
 - Estruturas de repetição



Python → estruturas de controlo

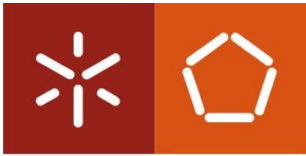
- Estruturas de decisão:
 - executa condicionalmente instruções
 - Permite definir um conjunto finito de cenários de execução, através da avaliação de condições

if condição :

Indentação

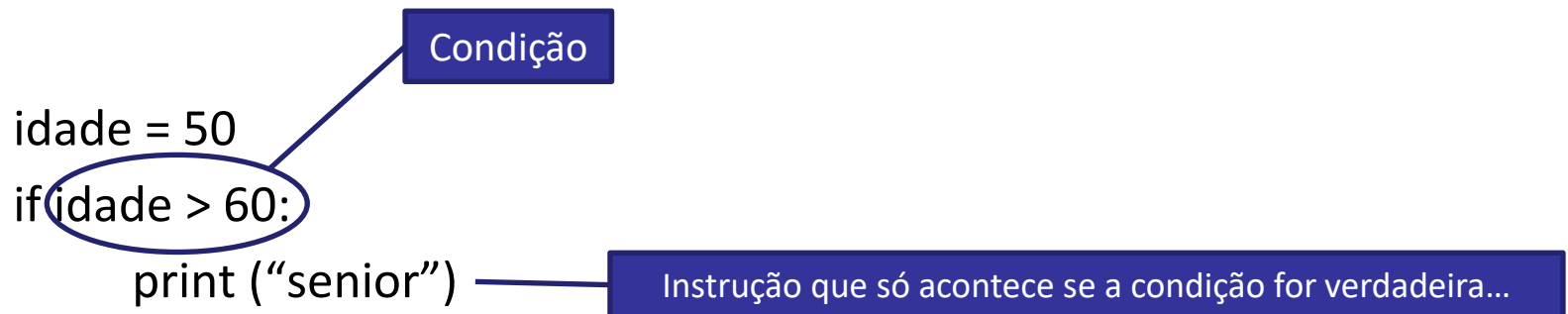
→ # faz os comandos aqui dentro do *if* se a condição for verdadeira

Este espaço, que se chama indentação, é de extrema importância!
Todas as instruções dentro do *if* tem de estar alinhadas usando a tecla TAB

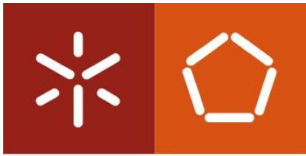


Python → estruturas de controlo

- Estruturas de decisão (exemplos):



O que será que acontece aqui?



Python → estruturas de controlo

- Estruturas de decisão (exemplos):

```
n = int(input("n?"))
```

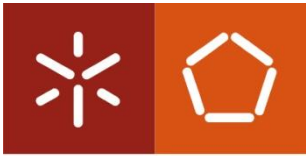
```
if n % 2 == 0:
```

```
    print ("número par")
```

Instrução que só acontece se a condição for verdadeira...

Condição
0

resto da divisão de n por 2 é 0 ?
(ou seja) número é par?



Python → estruturas de controlo

- Estruturas de decisão (exemplos):

#Exemplos de condições

`n > 0`

`contador < 0`

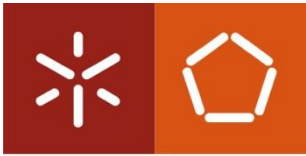
`itens == 10` `#igualdade`

`n + 1 != ultimo` `#diferença`

`nota < 9.5`

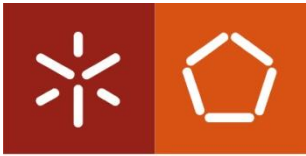
`18 < idade < 21` `#compreendido em intervalo`

`nome == "Camões"`



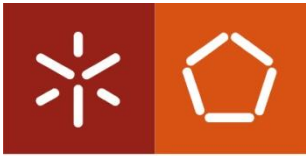
Python → estruturas de controlo

- Estruturas de decisão (condições compostas):
 - Podemos juntar condições habitualmente com os operadores lógicos **or** e **and**:
 - OR
 - resposta == "s" or resposta == "S" *#tornar uma resposta case-insensitive*
 - **A condição é verdadeira se uma das componentes de verificação for verdadeira!**
 - **A condição só é falsa quando todas as componentes de verificação são falsas!**
 - AND
 - nota1 >= 8 and nota2 >= 8 *#verificar se o aluno reúne os mínimos para ter nota*
 - **A condição é verdadeira se ambas as componentes de verificação forem verdadeiras!**
 - **A condição é falsa quando uma ou mais componentes de verificação são falsas!**



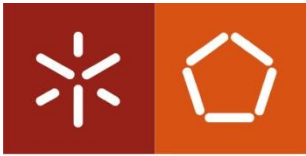
Python → estruturas de controlo

- Estruturas de decisão:
 - Condições com intervalos
 - No Python podemos escrever uma condição diretamente sob a forma de intervalo:
 $18 < \text{idade} < 21$
 - ...que é equivalente a escrever:
 $\text{idade} > 18 \text{ and } \text{idade} < 21$
 - Outras linguagem de programação usam só a última forma



Python → estruturas de controlo

- Estruturas de decisão (not):
 - temos ainda o operador:
 - **not** *condição*
 - **not** (nota1 >= 10 **and** nota2 >= 10)
 - Nega o resultado da verificação da parte da condição
 - Deve ler-se “se não ocorrer a condição...”
 - **not** *condição* é verdadeiro se condição for falsa
 - **not** *condição* é falso se condição for verdadeiro



Python → estruturas de controlo

- Estruturas de decisão → forma binária

if condição:

faz os comandos aqui dentro se a

condição for verdadeira

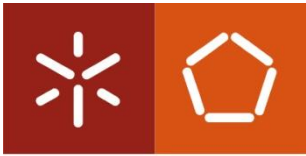
...

else:

se a condição for falsa faz os comandos

aqui dentro

...



Python → estruturas de controlo

- Estruturas de decisão → avaliação múltipla

if condição1:

faz os comandos aqui dentro se a condição1 for verdadeira ...

elif condição2:

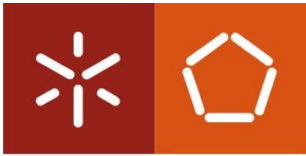
senão, se a cond.2 for verdadeira faz isto

elif condição3:

senão, se a cond.3 for verdadeira faz isto

else:

se nenhuma condição for verdadeira faz isto



Python → estruturas de controlo

- Estruturas de decisão

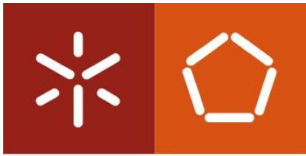
- podemos também atribuir o resultado de uma condição a uma variável:

```
idade = 17
```

```
maior_idade = idade >= 18
```

```
print(maior_idade)    # False
```

- maior_idade é uma variável do tipo boolean,
- experimentar `type(maior_idade)`



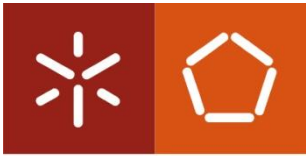
Python → estruturas de controlo

- Estruturas de decisão
 - Outra forma de escrever o programa anterior (mais extensa...):

```
idade = 17
```

```
if idade >= 18 :  
    maior_idade=True  
else:  
    maior_idade=False
```

```
print(maior_idade) # False
```

Python → estruturas de controlo

- Estruturas de decisão

- Podemos também usar a variável booleana na condição:

```
idade = 17
```

```
maior_idade = idade >= 18
```

```
if maior_idade: # equiv. a if maior_idade == True
```

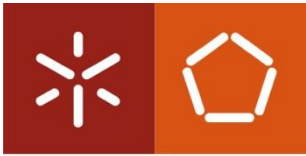
```
    print("Maior de idade")
```

```
else:
```

```
    print("Menor de idade")
```

- ... ou recorrendo ao operador ternário...

```
print("Maior de idade") if maior_idade else print("Menor de idade")
```



Python → estruturas de controlo

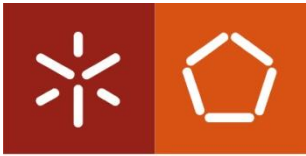
- Estruturas de repetição (**while**)
 - Repetição de instruções

while condição:

Indentação

repete o que está abaixo e dentro do ciclo
enquanto (while) a condição for verdadeira.

Lembram-se do tab para indentar? Na repetição (**ciclos**) também tem que se colocar. Tudo o que diz respeito à execução dentro do ciclo tem que estar a ele indentado!

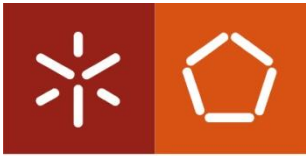


Python → estruturas de controlo

- Estruturas de repetição (**while**)

```
i = 0
while i < 10:
    # repete o que está dentro enquanto i < 10
    print ("aluno", i)
    i = i + 1    # acrescenta 1 a i

print("acabou") #quando chega aqui já saiu do ciclo
```



Python → estruturas de controlo

- Estruturas de repetição (**while**)

while True:

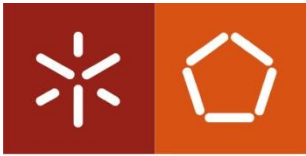
 # ciclo infinito!

 print ("eu sou o infinito")

while False:

 # ciclo não executa nenhuma iteração

 print ("nunca vou imprimir")



Python → estruturas de controlo

- Estruturas de repetição (**for**)

```
for i in range(n):
```

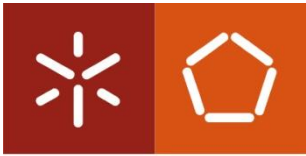
```
    # repete n vezes os comandos abaixo
```

```
    # a variável i guarda o número de repetições
```

Indentação

Lá está a indentação (TAB), outra vez, para o ciclo **for**...

- Vai repetir de i até n
- A variável i é atualizada a cada passagem (+1 no protótipo acima)
- Quando i chega a n, o ciclo para e o programa continua na primeira instrução que se encontrar a seguir ao ciclo, já fora dele.



Universidade do Minho

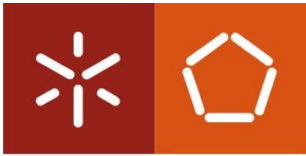
Python → estruturas de controlo

- Estruturas de repetição (**for**)

#Exemplo

```
for i in range(10):  
    print ("aluno", i)
```

```
print("acabou")
```

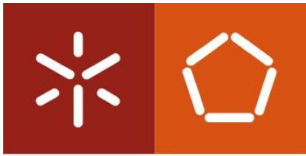


Python → Listas

- Guardar 10 números dados pelo utilizador
 - Como?
 - Se fizermos:

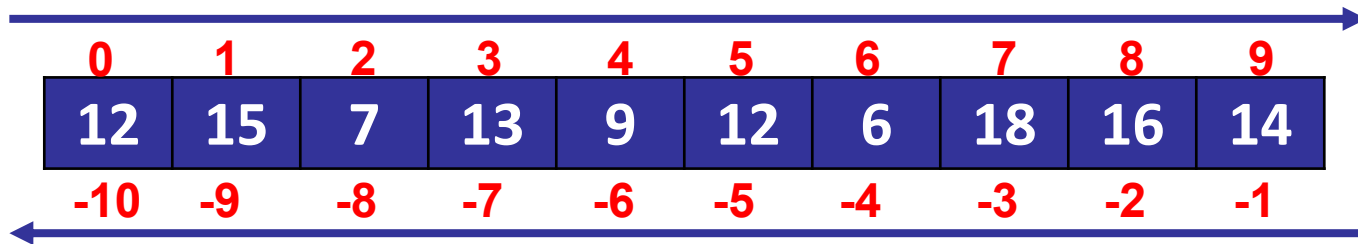
```
for i in range(10):  
    n = input("n", i, "?")  
# n no fim do ciclo só contém o último número introduzido  
# perdemos os números anteriores
```
 - Com o que aprendemos até agora:

```
n1 = input("n1?")  
n2 = input("n2?")  
...  
n10 = input("n10?")
```

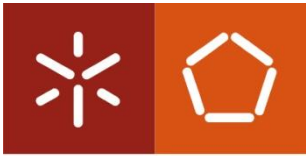


Python → Listas

- `l = [12, 15, 7, 13, 9, 12, 6, 18, 16, 14]`

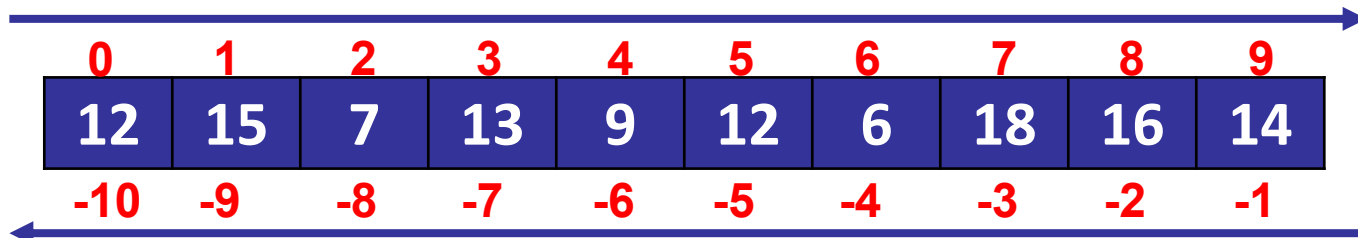


- `l[0]` # 12, o 1º elemento da lista
- `l[9]` # 14, último elemento da lista
- `len(l)` # tamanho da lista, 10 elem
- `l[-10]` # o 1º elemento da lista
- `l[-1]` # o último elemento da lista

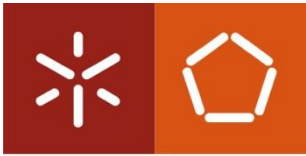


Python → Listas

- Exemplos de listas em Python



- `vazia = []` # lista vazia
- `lista = [3.4, 2.1, 1.6]` # lista de números
- `nomes = ["maria", "camões", "joana"]` # lista de strings
- `matriz = [[1, 0, 0], [0, 1, 0], [0, 0, 1]]` # lista de listas (matriz)



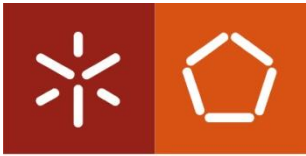
Python → Listas

- Imprimir Listas

- `nomes = ["maria", "camões", "joana"]` `#Criar lista`
 - `print(nomes)` `#Imprime ["maria", "pedro", "joana"]`

- Substituir (editar) elemento

- `nomes = ["maria", "luis", "joana"]`
 - `nomes[1] = "camões"`
 - `print(nomes)` `#Imprime ["maria", "camões", "joana"]`



Python → Listas

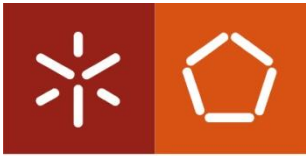
- Criar listas inicializadas

```
list(range(10))          #[0,1,2,3,4,5,6,7,8,9]
list(range(1,11))        #[1,2,3,4,5,6,7,8,9,10]
list(range(0,30,5))      #[0,5,10,15,20,25]
```

```
range(stop)
range(start, stop[, step])
```

Nota 1: range só funciona com valores inteiros!

Nota 2: o arange do *numpy* suporta extremidades e saltos com *float*



Python → Listas

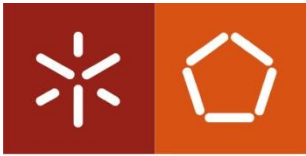
- Percorrer elementos de uma lista

usa-se o ciclo **for** para percorrer listas

```
idades = [12, 13, 14, 15]
```

```
for i in idades:
```

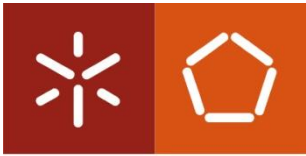
```
    print(i)
```



Python → Listas

- Percorrer elementos de uma lista
 - Dizer “Olá” a todos os nomes de uma lista:

```
# usa-se o ciclo for para percorrer listas  
lista = ["maria", "camões", "joana"]  
for nome in lista:  
    print("Olá", nome)
```



Python → Listas

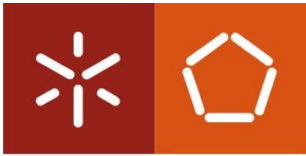
- Inserir/remover elementos de uma lista

`lista.append(7)` # acrescenta 7 no fim

`lista.insert(1, 1.7)` # insere na pos 1, 1.7 → (índice, elemento)

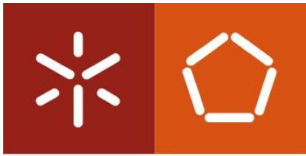
`del lista[2]` # apaga o 3º elem. (pos 2)

- Nota: a lista tem de existir previamente, antes de acrescentar ou apagar elementos



Python → Listas

- Funções especiais que podem ser usadas com as listas Python:
 - `min (lista)` # valor mais pequeno da lista (numérico ou alfabético)
 - `max (lista)` # valor maior da lista (numérico ou alfabético)
 - `lista.sort()` # ordena a lista
 - `lista.reverse()` #troca a ordem dos elementos (reverte)



Python → Listas

- Utilização da palavra reservada “in”:

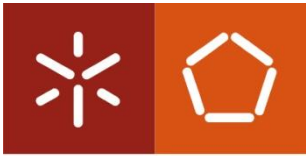
```
lista = ["maria", "camões", "joana"]
```

```
if "camões" in lista:
```

```
    print ("camões está inscrito")
```

```
else:
```

```
    print ("camões não está inscrito")
```

Python → Listas

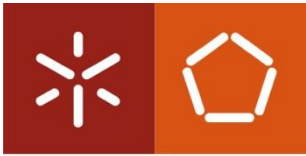
- Concatenar (juntar) 2 listas

```
lista1 = [3.4, 2.1, 1.6]
```

```
lista2 = [4, 1, 2]
```

```
listajunta = lista1 + lista2      # junta as 2 listas
```

```
print(listajunta)
```



Universidade do Minho

Python → Listas

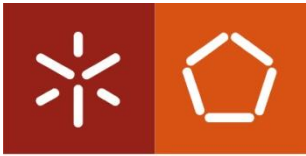
- Fatiar listas

```
my_list = ['p', 'y', 't', 'h', 'o', 'n']
```

```
print(my_list[2:5])    # ['t', 'h', 'o']
```

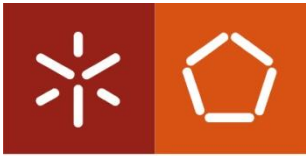
```
print(my_list[:-3])    # ['p', 'y', 't']
```

```
print(my_list[3:])     # ['h', 'o', 'n']
```



Python → Listas

- List comprehensions: notação para criar listas de forma mais concisa.
- Assemelha-se à notação matemática para definir conjuntos, por exemplo:
 - $S = \{x^2 : x \text{ in } \{0 \dots 9\}\}$
 - $V = (1, 2, 4, 8, \dots, 2^{12})$
 - $M = \{x \mid x \text{ in } S \text{ and } x \text{ even(par)}\}$



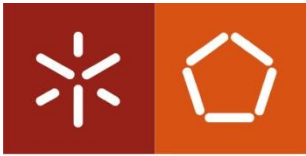
Python → Listas

- List Comprehensions: notação concisa. para criar listas de forma mais

— $S = \{x^2 : x \text{ in } \{0 \dots 9\}\}$
— $V = (1, 2, 4, 8, \dots, 2^{12})$
— $M = \{x \mid x \text{ in } S \text{ and } x \text{ (par)}\}$

... em Python...

— $S = [x**2 \text{ for } x \text{ in range}(10)]$
— $V = [2**i \text{ for } i \text{ in range}(13)]$
— $M = [x \text{ for } x \text{ in } S \text{ if } x \% 2 == 0]$

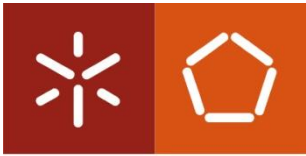


Universidade do Minho

Python → Listas

- List Comprehensions
 - Para fazer uma lista com os valores da função seno entre 0 e 2π

```
s = [math.sin(x*math.pi/180) for x in range(360)]
```



Python → Listas

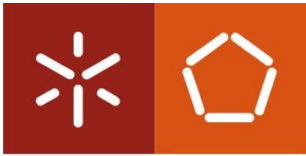
- List Comprehensions

- Exercício: fazer uma lista com os valores da função seno entre 0 e 2π

```
s = [math.sin(x*math.pi/180) for x in range(360)]
```

calcula o seno,
x é convertido para
radianos<

x varia entre 0 e 359 graus;
range só aceita inteiros
portanto,
usou-se graus p/ definir
conjunto



- List Comprehensions

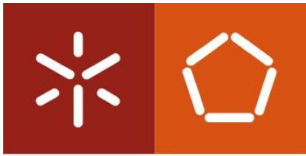
```
s = [math.sin(x*math.pi/180) for x in range(360)]
```

#equivalente a...

```
s = []
```

```
for x in range(360):
```

```
    s.append(math.sin (x*math.pi / 180))
```



Python → Listas

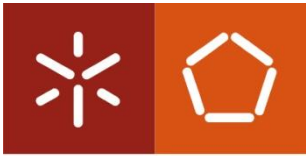
- List Comprehensions

```
naipe = ["ouros", "paus", "espadas", "copas"]
```

```
valor = [2, 3, 4, 5, 6, 7, 8, 9, 10, "Valete", "Dama", "Rei", "Às"]
```

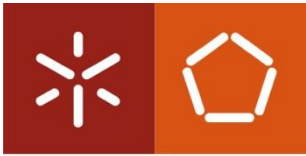
#gerar o baralho completo numa lista:

```
baralho = [ [i, j] for i in valor for j in nape]
```

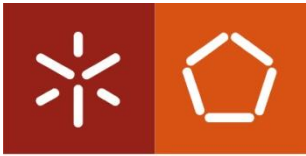
Python → Tuplos

- Em tudo semelhante às listas, mas são imutáveis (não permitem alterações)
- Usam-se quando está envolvida uma lista que não é suposto ser alterada durante execução do programa
 - `l = (12, 15, 7, 13, 9, 12, 6, 18, 16, 14)` #nos tuplos usa-se () em vez de []
ou
 - `l = 12, 15, 7, 13, 9, 12, 6, 18, 16, 14` #também é possível definir tuplos usando apenas virgulas, sem delimitadores nos extremos (sem () ou []).
 - `l[0]` representa o 1º elemento do tuplo, `l[9]` representa o último (como nas listas)



Python → Tuplos

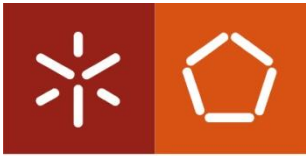
- `l = (12, 15, 7, 13, 9, 12, 6, 18, 16, 14)`
ou
- `l = 12, 15, 7, 13, 9, 12, 6, 18, 16, 14`
- Regras:
 - `append / del` não funcionam com tuplos, porque não podem ser mudados.
 - `v = 1.5, 0, -2` `#define um tuplo.`
 - `x, y, z = v` `# as variáveis x,y,z ficam com os elementos respetivos do tuplo v`



Python → Strings

- Tipo de variável que guarda sequências de caracteres.

```
s = "abcde"    # criar a string...  
               # pode usar-se "..." ou '...'  
  
print(s)  
print( s[0] )  # "a" 1º character  
print( s[-1] ) # "e" último character  
print( s[0:3] ) # fatiar do char 0 ao 3 → "abc"  
  
print( s[0:5:2] ) # fatiar do char 0 ao 5, de 2 em 2 posições → "ace"  
  
print(len(s))  # comprimento da string, incluindo espaços → 5
```



Python → Strings

- Regras

- Uma vez criada, não é possível mudar o conteúdo da *string*.

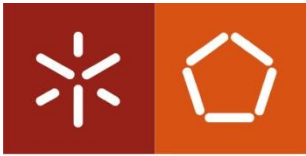
- As *strings* são imutáveis!

- Por exemplo tentar mudar a primeira letra dá erro:

```
s="abcde"
```

```
s[0]="z" # Dá erro
```

- ...por esta razão as várias operações para manipular *strings* como *replace*, *upper* e *lower* não modificam a *string* mas criam uma nova!



Python → Strings

- Funções de transformação rápida

```
s = "Eduardo Madeira"
```

```
new = s.replace(" ", "_")
```

```
# substitui o espaço por _ numa nova string new
```

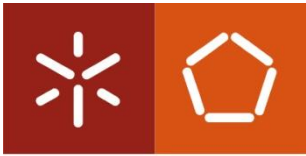
```
print(s)
```

```
# 'Eduardo Madeira'
```

```
print(new)
```

```
# 'Eduardo_Madeira'
```

...*replace* não muda a *string* mas retorna uma nova *string*, porque as *strings* são imutáveis!



Python → Strings

- Funções de transformação rápida

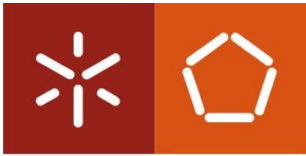
```
s = "Eduardo Madeira"
```

```
new = s.lower() # converte a string para lowercase (letras minúsculas)
```

```
print(s) # 'Eduardo Madeira'
```

```
print(new) # 'eduardo madeira'
```

...*lower* não muda a *string* mas retorna uma nova *string*, porque as *strings* são imutáveis!



Python → Strings

- Funções de transformação rápida

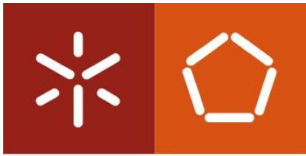
```
s = "Eduardo Madeira"
```

```
new = s.upper() # converte a string para uppercase (letras maiúsculas)
```

```
print(s) # 'Eduardo Madeira'
```

```
print(new) # 'EDUARDO MADEIRA'
```

...*upper* não muda a *string* mas retorna uma nova *string*, porque as *strings* são imutáveis!



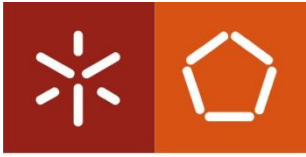
Python → Strings

- Funções de transformação rápida

```
s = "Eduardo Madeira"
```

```
l = s.split(" ")           # divide a string pelo caracter indicado  
                           # retorna uma lista de strings
```

```
print(l)                   # ['Eduardo', 'Madeira']
```

Universidade do Minho

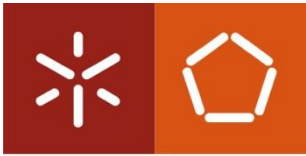
Python → Strings

- Funções de transformação rápida

```
l = ['Eduardo', 'Madeira']
```

```
s = "_".join(l) # junta uma lista de strings numa nova string com o  
                # caracter indicado
```

```
print(s)        # Eduardo_Madeira
```



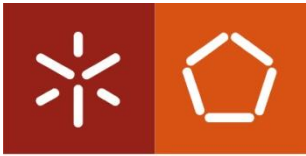
Python → Strings

- Funções de transformação rápida

```
s = "abcde"
```

```
l = list(s)      # converte a string numa lista
```

```
print(l)        # ['a', 'b', 'c', 'd', 'e']
```



Python → Strings

- Operações com *strings*, usando sinais aritméticos

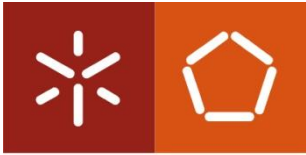
```
s1 = "Eduardo"
```

```
s2 = "Madeira"
```

```
s = s1 + s2      # junta as strings numa com o conteúdo "EduardoMadeira"
```

```
s = s1 + " " + s2  # junta as strings numa com o conteúdo "Eduardo Madeira"
```

```
s = "-" * 10      # ----- (10x)
```



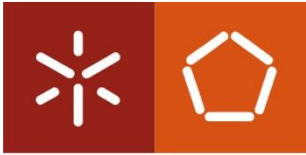
Python → Strings

- Verificar se uma determinada *substring* está presente na *string* (operador *in*):

```
s = "abcde"
```

```
if "a" in s:
```

```
    print ("s contém um a")
```



Python → Strings

- Iterar sobre uma string: semelhante a iterar uma lista, usando uma estrutura de repetição (por exemplo, *for*)

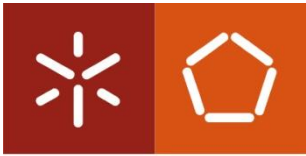
programa para soletrar:

S = "BRUNO"

for c in S:

 # para cada iteração c contém um caracter da *string*

 print(c)



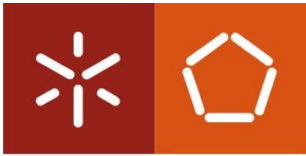
Universidade do Minho

Python → Números aleatórios



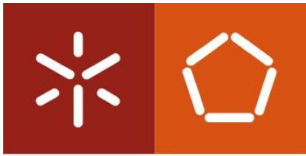
- Há várias situações onde programas precisam de recorrer a gerador de números aleatórios (números "à sorte")
- Exemplos:
 - um jogo que sorteia uma mão de cartas.
 - métodos numéricos de cálculo integral (e.g método monte-carro)
 - encriptação de dados





Python → Números aleatórios

- Número aleatório é um número que pertence a uma série numérica que não pode ser previsto a partir dos números anteriores.
- O computador não consegue gerar números verdadeiramente aleatórios (porque é uma máquina determinística) mas pode gerar sequências "imprevisíveis" a partir de um número inicial (seed) e um algoritmo (pseudo-)aleatório.
- **O módulo random do Python:** contém várias funções para escolher números (pseudo) aleatórios.
 - `import random`
 - <https://docs.python.org/3.7/library/random.html>



Python → Números aleatórios

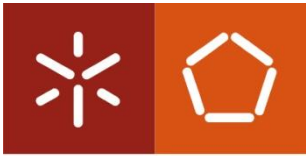
- Seleção de número aleatório do tipo float
 - `random.random()` `# 0 <= N < 1`
 - `random.uniform(a, b)` `# a <= N <= b`
- Seleção de número aleatório do tipo int
 - `n = random.randint(a, b)` `# a <= n <= b`
- seleção de um elemento aleatório:
 - `random.choice(l)` `# escolhe um elemento aleatório de l`

l pode ser uma **lista**:

```
l = ["a", "b", "c"]  
e = random.choice(l) # e =  
a ou b ou c
```

l pode ser um **range**:

```
n =  
random.choice(range(0,20,2))  
# escolhe um elemento  
aleatório dos números  
# 0, 2, 4 ..18
```

Python → Números aleatórios

- **Baralhar** uma lista:

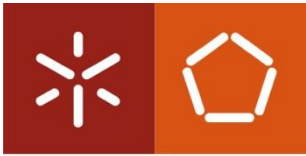
```
l = [1, 2, 3, 4, 5]
random.shuffle(l)      # lista l fica baralhada
print(l)
```

- **Amostrar** elementos aleatórios de uma lista

```
l = random.sample(p, k)    # cria uma nova lista com k elementos aleatórios de p,
                           # em que p pode ser uma lista ou um range.
```

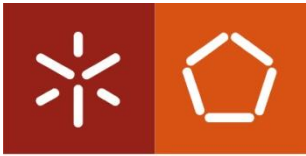
- **Exemplo para amostrar** elementos aleatórios de uma lista

```
l = random.sample(range(1000),10)    # cria uma lista l com 10 elementos aleatórios
                                     # entre 0 e 999
```



Python → Números aleatórios

- Pode ser útil no início do programa usar a função `seed`.
`random.seed()`
- Garante que o gerador de números aleatórios é reiniciado e a sequência gerada será diferente.
- Se passarem um número (qualquer), o gerador de números aleatórios será inicializado sempre da mesma forma.
- Útil quando estão a fazer *debug* ao programa e querem garantir que vão obter os mesmos números aleatórios.



Python → Números aleatórios

- Exemplo com o jogo da roleta: Jogador aposta um número entre 1 e 36 e o valor da aposta. Sortear um número entre 0 e 36, diz se jogador ganhou ou perdeu.



import random

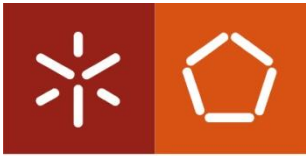
```
numero = int(input("aposta num número entre 1 e 36 ?"))  
while not (1 <= numero <= 36):  
    print("número inválido")  
    numero = int(input("aposta num número entre 1 e 36 ?"))
```



```
print("a girar a roleta...")  
sorteio = random.randint(0,36)  
print("saiu o número", sorteio)  
if sorteio == numero:
```

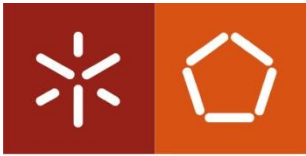


```
    print("GANHOU !")  
else:  
    print("... perdeu...")
```



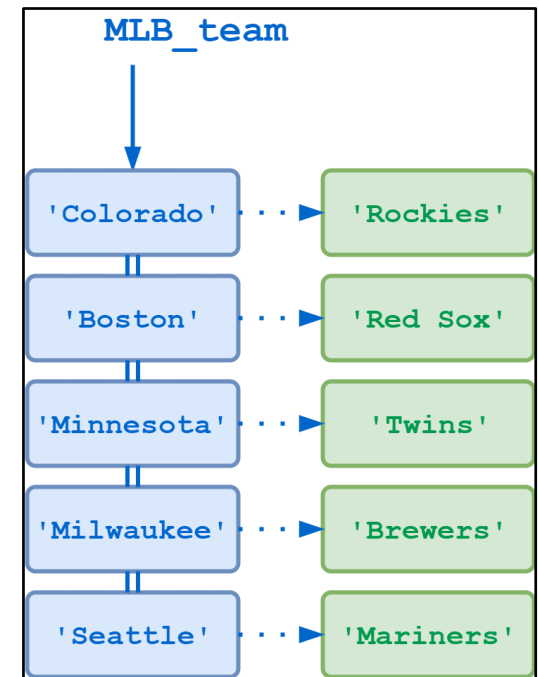
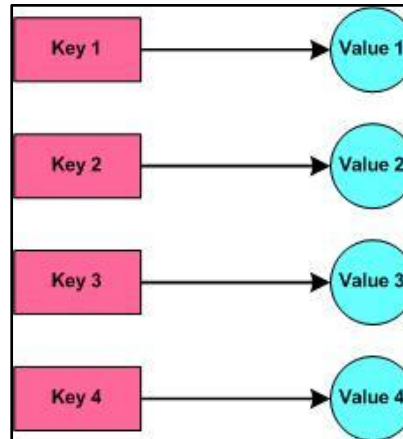
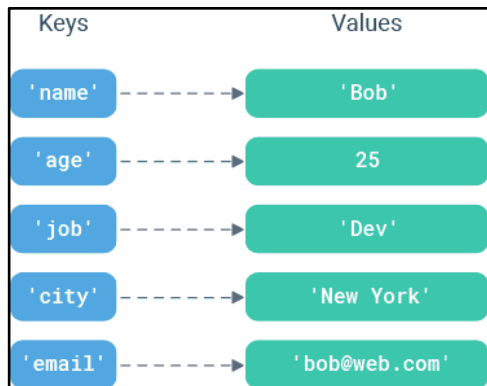
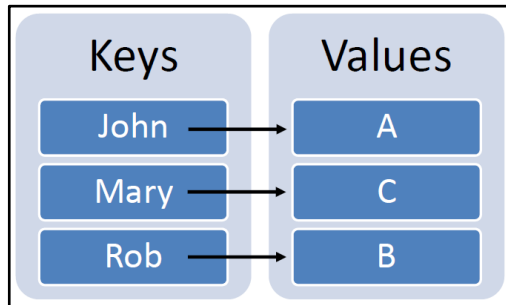
Python → Dictionary (dicionários)

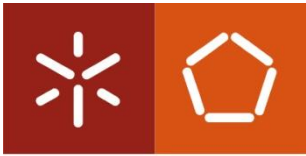
- É uma outra estrutura de dados que, tal como as listas, permitem-nos guardar um conjunto de dados.
- Os dados são conservados no formato *key:value*
- O PHP (outra linguagem de programação), implementa uma estrutura muito parecida, que se designa de *associative array*
- A partir do Python 3.6 a ordem de inserção é conservada



Universidade do Minho

Python → Dictionary (dicionários)



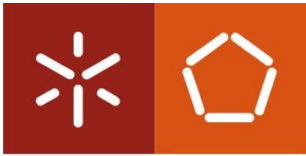


Python → Dictionary (dicionários)

- Referimo-nos a cada elemento guardado nas listas com um número (índice):

```
alunos= ["Pedro", "Ana", "Maria", ...]  
         0       1       2       ...  
  
alunos[0]  
alunos[1]  
...
```

Diagram illustrating list indexing. The word "índice" is written in red, with two lines pointing to the indices 0 and 1 in the list access examples below the list definition.

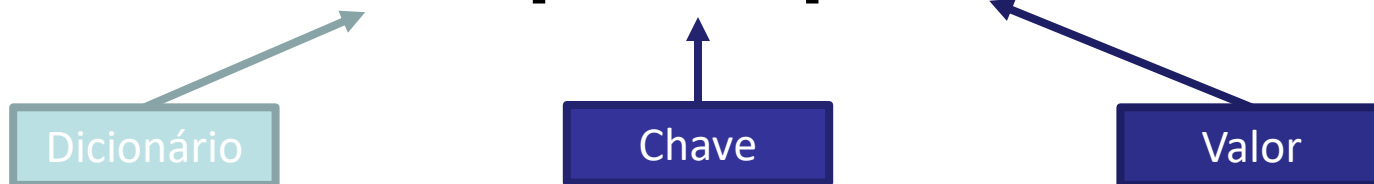


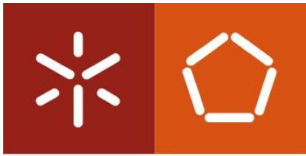
Python → Dictionary (dicionários)

- Nos dicionários indexamos os elementos por qualquer tipo de variável, em particular, e o mais usado, uma *string*.

```
alunos["a12345"] = "José"
```

```
alunos["a87654"] = "Ana"
```





Python → Dictionary (dicionários)

- Criar um dicionário:

d = {chave1: valor1, chave2: valor2, ... }

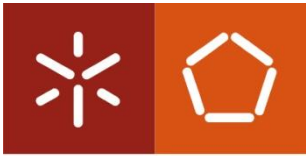
- Exemplos:

```
produtos = {"XPT01" : "Smartphone X", "XPT02" : "Laptop Y"}
```

```
d = {}      # dicionário vazio
```

```
d = {"a123456": "José Fernandes", "a234567": "Ana Pinto" }
```

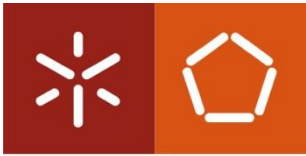
```
morse = {"A": ".-", "B": "-...", "C": "-.-." }
```

Universidade do Minho

Python → Dictionary (dicionários)

- Acrescentar um elemento ao dicionário:
`produtos["XPT03"] = "Tablet Z"`
- Remover um elemento do dicionário:
`del produtos["XPT03"]`
- Verificar a existência de uma chave no dicionário:
`if "XPT03" in produtos:`
...



Python → Dictionary (dicionários)

- Iterar sobre as chaves de um dicionário:

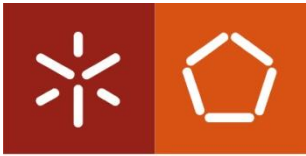
```
for id in produtos:
```

```
    print("O produto de código", id, "designa-se de", produtos[id])
```

- Iterar sobre as chaves e valores de um dicionário:

```
for id, produto in produtos.items():
```

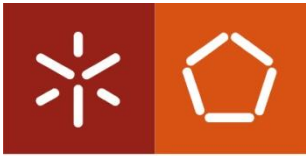
```
    print("O produto de código", id, "designa-se de", produto)
```



Python → Dictionary (dicionários)

- Exemplo:

```
dict_produtos = {  
    "XPT01" : Computador A,  
    "XPT02" : Smartphone B,  
    "XPT03" : Tablet QWERT,  
    "XPT04" : Teclado XPTO,  
    "XPT05" : Wi-Fi router C,  
}  
  
for produto_id in dict_produtos :  
    print("ID: ", produto_id, "; NOME: " dict_produtos[produto])
```



Universidade do Minho

Python → Funções

Até agora usamos funções já definidas no Python, como por exemplo:

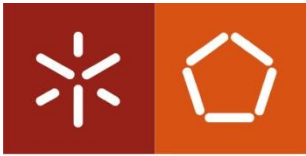
`print()`, `input()`, ...

...ou funções já definidas em módulos externos:

`random.randint()`

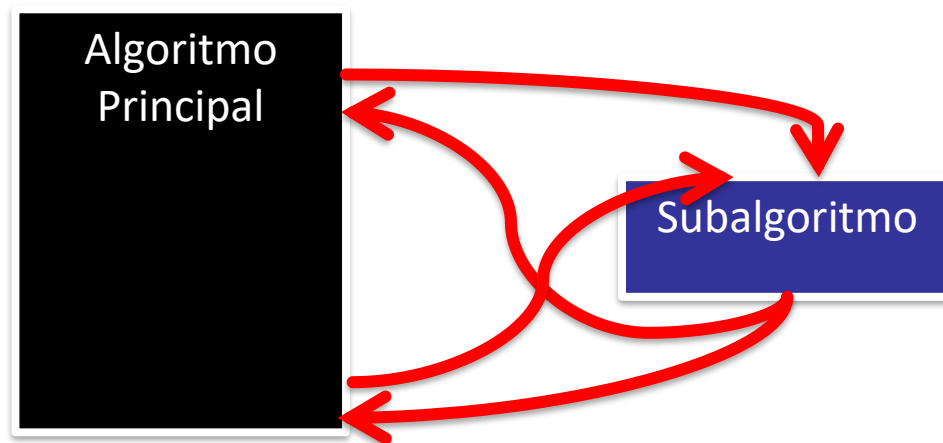
`math.sqrt()`

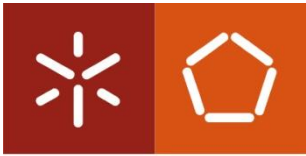
Hoje, vamos passar agora a criar e usar também **as nossas funções**.



Python → Funções

Funções definem um bloco de código, separado do programa principal que só é executado quando é chamado.



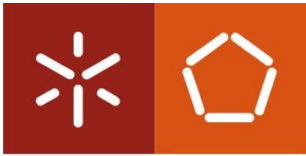


Universidade do Minho

Python → Funções

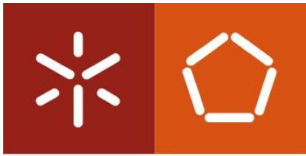
```
def minhafuncao():  
    print("entrei na função")  
...  
minhafuncao()  
...
```

só nesta altura o código dentro da função é executado...



Python → Funções

- **Porque é que são importantes?**
 - Permitem organizar o código de forma mais modular.
 - Evita repetições de código, posso simplesmente usar a função sempre que precisar.
 - Posso mudar o código de uma função sem mudar o resto do programa.

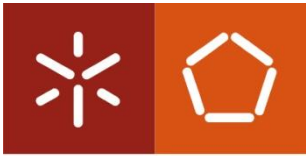


Python → Funções

- **Aplicação:** um programa que peça um número ao utilizador, faça um cálculo complicado e imprima resultado.

Até agora escreveriam o programa desta forma:

```
n = int(input("n?"))  
# aqui vão ter muitas linhas de código  
# para fazer um cálculo complicado  
# ...  
  
print(resultado)
```

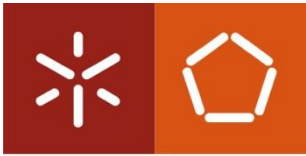



Python → Funções

- Podem passar a escrever da seguinte forma (simplificar, tornar conciso, reutilizar):

```
def calcula_complicado (v):  
    # aqui vão ter muitas linhas de código  
    # para fazer um cálculo complicado  
    # ...  
    return r # passa de volta o resultado
```

```
n = int(input("n?"))  
resultado = calcula_complicado (n)  
print(resultado)
```



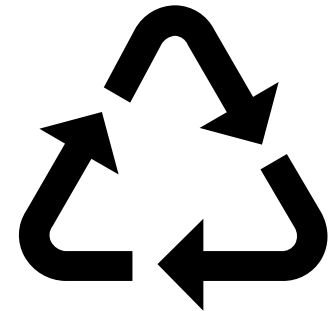
Python → Funções

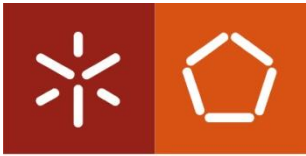
- Se for necessário ao programa fazer outro cálculo mais à frente podem simplesmente **chamar** novamente a **função**:

```
n = int(input("n?"))  
resultado = calculo_complicado(n)  
print(resultado)
```

...

```
n2 = int(input("n2?"))  
resultado2 = calculo_complicado(n2)  
print(resultado2)
```



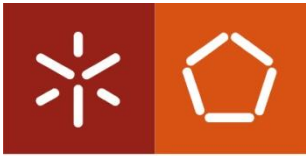


Python → Funções

- Se for houver um erro no cálculo complicado, corrigem só a função e o resto do programa fica igual (manutenção simplificada):

```
def calculo_complicado (v):  
    # novas linhas de código  
    # para fazer um cálculo complicado  
    # ...  
    return r  
  
# o resto do programa fica igual  
...  
resultado = calculo_complicado(n)  
...  
resultado2 = calculo_complicado(n2)
```





Python → Funções

- Como fazer uma função?

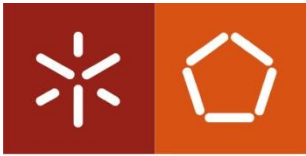
podem usar um nome qualquer, por
convenção usem nomes minúsculos

Palavra reservada “def” Nome função Parâmetros

```
def minhafuncao():
```

Corpo da
função

```
    #código da função aparece indentado  
    #aqui dentro  
    print("minha função")
```

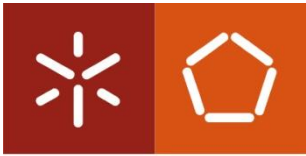


Python → Funções

- Chamamos a função sempre que necessitamos que esse código seja executado

```
def minhafuncao():  
    print("entrei na função")  
...  
minhafuncao()  
...
```

só nesta altura o código dentro da função é executado...



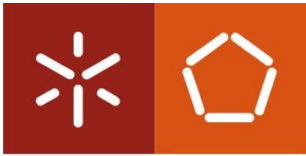
Python → Funções

- Função com argumento

Nome função Parâmetros

```
def cumprimenta(nome):  
    """Esta função cumprimenta o nome  
    passado como argumento"""  
    print("Olá", nome, "!")
```

```
cumprimenta("José")  
cumprimenta("Ana")
```



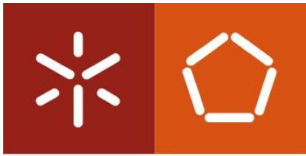
Python → Funções

- A função pode retornar um resultado

```
def minhafuncao():  
    print("entrei na função")  
    return "resultado"  
2  
↓ ...  
a = minhafuncao()  
# a fica com "resultado"
```

1

2



Python → Funções

- Função com argumentos e retorno

```
def media(a, b):
```

```
    # a e b contém o valor passado como argumento
```

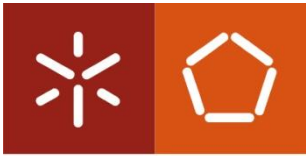
```
    c = (a+b) / 2
```

```
    return c
```



Retorna o resultado do
cálculo

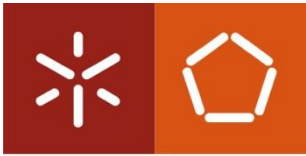
```
m = media(10, 20) # chama a função acima e m fica  
                  # com o resultado da media
```

Python → Funções

- Retornar múltiplos valores

```
def calculo_pos_projetil(x0, y0, vel, angle, t)
    """ x0 e y0 são a posição inicial de lançamento, vel a
    velocidade, o angulo em radianos, t o tempo"""
    x = x0 + vel*math.cos(angle) * t
    y = y0 + vel*math.sin(angle) * t + 0.5*(-10)*t**2
return x, y # retorna um "tuplo" com x e y
```



Python → Funções

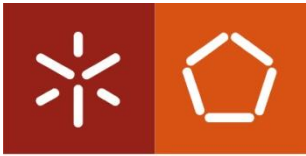
- scope* das variáveis

```
def f():  
    a = 1  
    return a
```

} **variáveis criadas dentro da função só podem ser usadas aqui dentro**

```
f()  
print(a) # isto dá erro! não conhece variável a fora da função
```

* scope das variáveis = contexto das variáveis



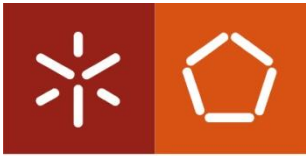
Python → Funções

- scope* das variáveis

`a = 5` **}** variável "a" externa à função

`def f():`
 `a = 2` **}** variável com o mesmo nome dentro da função,
 mas diferente da variável "a" externa

`f()`
`print(a)` # imprime 5



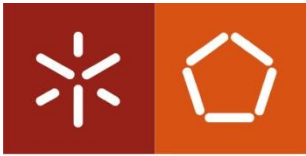
Python → Funções

- scope* das variáveis

`a = 5` **}** variável "a" externa à função

`def f():`
 global a **}** Com a *keyword* "global" antes da variável, estamos a referir-mo-nos à variável externa ("a" interno = "a" externo)...
 `a = 1`

`f()`
`print(a)` # imprime 1

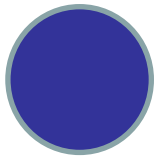


Universidade do Minho

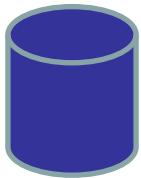
Python → Funções

- Uma função pode chamar outra função

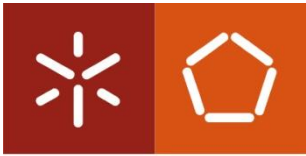
```
import math
```



```
def areaCirculo(raio):  
    return math.pi * raio**2
```



```
def volumeCilindro(altura,raio):  
    return altura * areaCirculo(raio) # chama a função anterior para calcular a  
                                     # área e multiplicar pela altura
```



Python → Funções

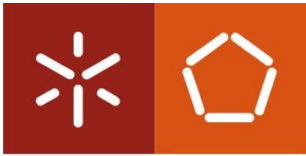
- Boas práticas
 - Dar às funções um nome sugestivo → que permita perceber o que fazem.
 - Escrever um comentário de ajuda a explicar como se usa a função.

def primo(numero):

```
    """ função que dado um numero inteiro positivo retorna True se é primo ou  
    False se não """
```

```
    ...
```

```
    return p
```



Python → Funções

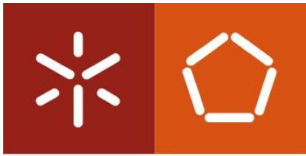
- Boas práticas
 - Uma boa função **deve poder ser o mais flexível possível** para poder ser usada em diferentes situações:

```
def calculaAreaCirculo(raio):
```

```
    # esta função imprime a área
```

```
    print(math.pi * raio**2)
```

```
calculaAreaCirculo(1) # ... se só imprime não posso usar o resultado em cálculos !
```



Universidade do Minho

Python → Funções

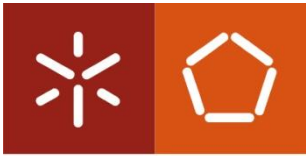
- Boas práticas

```
def calculaAreaCirculo(raio):
```

```
    return math.pi * raio**2    #mais geral, retorna o valor em vez de imprimir
```

```
print (calculaAreaCirculo(1))    # posso imprimir
```

```
b = calculaAreaCirculo(1) * 3    # mas também usar numa fórmula
```

Python → Funções

- como devem estruturar o programa com funções:

```
import xyz  
import ...
```



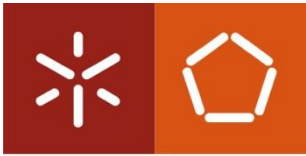
Fazer os *imports* todos no início do programa

```
def funcao1(a):  
...  
def funcao2(a):  
...  
def funcao3(a):  
...
```



Definir apenas as funções necessárias!

```
# programa principal começa a partir daqui  
...
```



Python → Funções

- Curiosidade - função recursiva (*)

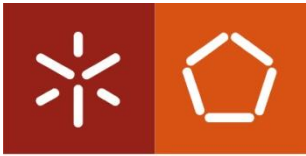
Em matemática, posso definir uma função de forma recursiva*, por exemplo o fatorial:

$$0! = 1$$

$$n! = n * (n-1)!$$

fatorial de n é definido à custa de fatorial de $n - 1$

*função recursiva é a aquela que se chama a si própria

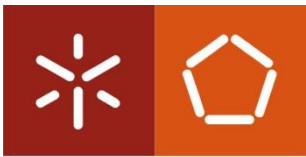


Python → Funções

- Função recursiva
 - também posso ter uma função em Python que se chame a si própria

```
def fatorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * fatorial(n-1) # chama a própria função
```

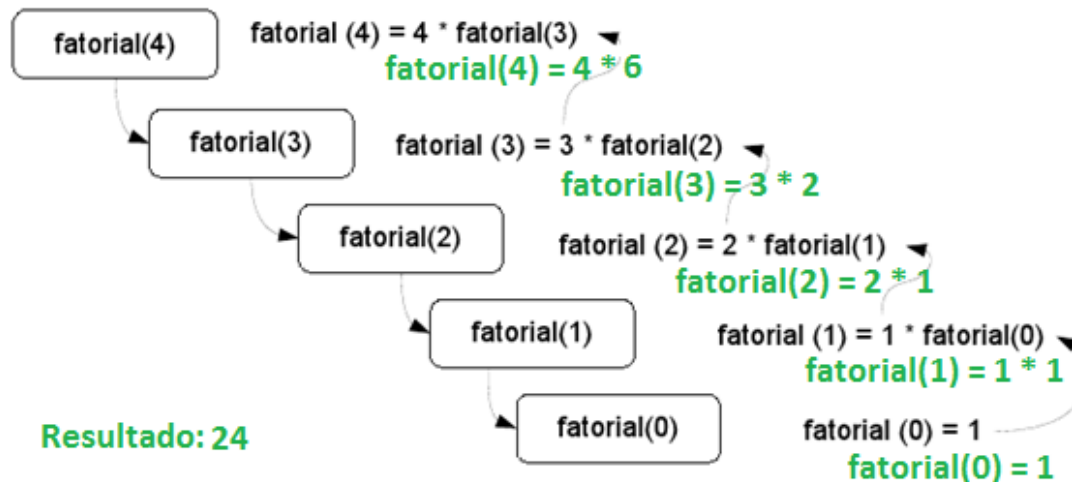
fatorial(3)

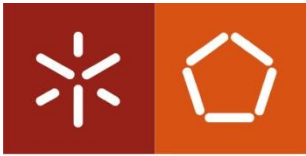


Python → Funções

- Função recursiva...

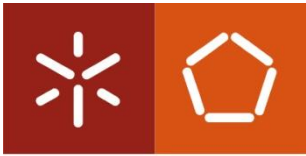
*Lembrem-se que a leitura é de baixo para cima!





Python → Ficheiros

- Forma simples de armazenar dados no computador.
- Tipos:
 - Texto: Guardam sequências de caracteres e linhas sem formatação (*plain text*) que podem ser lidas diretamente num qualquer editor.
 - Binário: Em oposição a ficheiros de texto, usam alguma codificação de dados obedecendo a algum formato específico (por exemplo imagens, música,...) só podendo ser manipulados com programas desenvolvidos para o efeito.

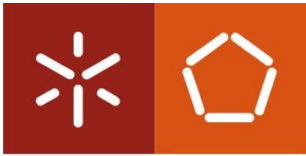


Universidade do Minho

Python → Ficheiros

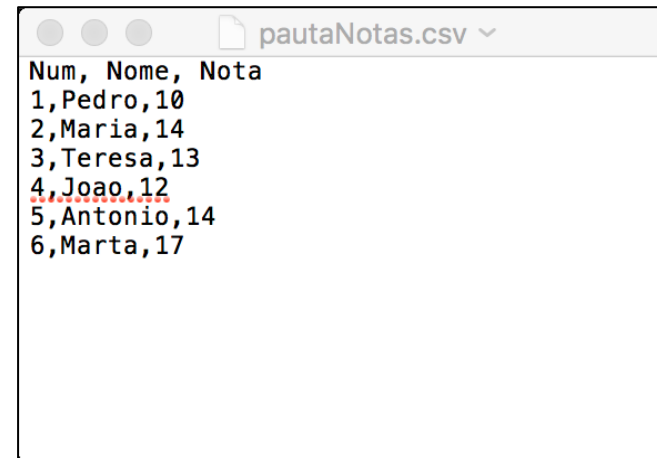
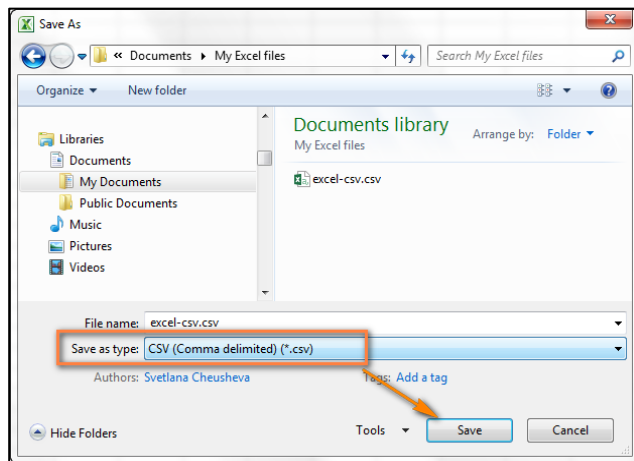
- Para distinguir de outros formatos, os ficheiros de texto usam como extensão `.txt` ou `.csv`
- Podemos criar / editar ficheiros de texto no *notepad* ou outro editor desde que gravados como *plain text*

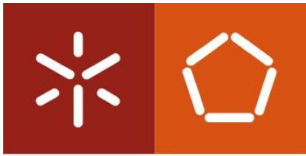




Python → Ficheiros

- O Excel permite gravar folhas de cálculo como ficheiro de texto .csv (*comma separated values*)





Python → Ficheiros

- Exemplo ficheiro .csv

```
Num;Nome;Nota
```

```
1;A;10
```

```
2;B;11
```

```
3;C;12
```

```
4;D;13
```

```
5;E;14
```

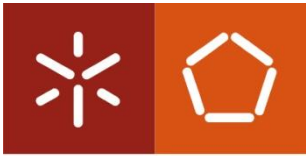
```
6;F;15
```

```
7;G;16
```

```
8;H;17
```

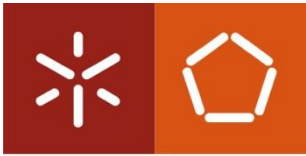
Nota Apesar do nome, um ficheiro csv pode usar qualquer caracter para separação dos valores para além da “,” como por exemplo, espaço, “;” TAB ...

- Vários programas permitem exportação/importação de dados como ficheiro csv



Python → Ficheiros de texto

- Encoding: Ascii vs utf-8
 - Nos ficheiros de texto, tradicionalmente cada caracter (letra, número, espaço...) é guardado como 1 byte (8 bits) havendo um total de 256 caracteres possíveis (ASCII) → a..z, A-Z, 0-9, outros símbolos.
 - Com o suporte de múltiplas línguas cada caracter é hoje codificado como 4 bytes (utf-8) → caracteres especiais multilingue.



Python → Ficheiros de texto

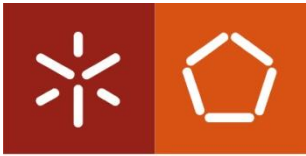
- `variável = open(nomedoficheiro, modo)`

— Exemplos:

```
f = open('oMeuFicheiro.txt', 'r')
```

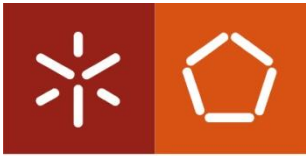
```
f = open('/users/foo/oMeuFicheiro.txt', 'r')
```

pode incluir também o caminho para o ficheiro



Python → Ficheiros de texto

- `f = open('oMeuFicheiro.txt', 'r')`
- Modos de acesso possíveis:
 - `r` - abre em modo leitura (**r**ead)
 - `w` - abre em modo escrita apagando o que está lá (**w**rite)
 - `a` - abre em modo de escrita para acrescentar (**a**ppend)
 - `r+` - abre em modo de leitura e escrita



Universidade do Minho

Python → Ficheiros de texto

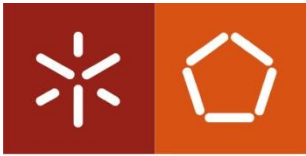
```
f = open('oMeuFicheiro.txt', 'r')
```

...

```
f.close()
```

No fim de ler/escrever o ficheiro devem fechar com o **close()**

Depois de **fechar** o ficheiro posso-o **abrir novamente** se necessário.



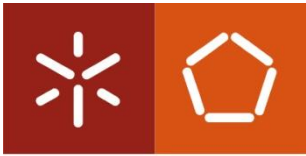
Python → Ficheiros de texto (leitura)

- Comandos para leitura:

`f.read()` # lê todas as linhas de um ficheiro para uma *string*

`f.readlines()` # lê todas as linhas de um ficheiro para uma lista

`f.readline()` # lê a próxima linha de o ficheiro para uma *string*

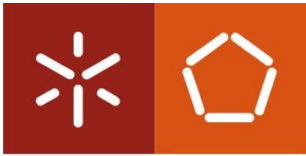


Python → Ficheiros de texto (leitura)

- Exemplo - leitura integral:

```
f = open('oMeuFicheiro.txt', 'r')
s = f.read()    # lê o conteúdo todo do f para a string s
                # depois do primeiro read qualquer outro
                # read retornaria uma string vazia
                # porque o ficheiro já foi todo lido

print(s)
f.close()
```



Python → Ficheiros de texto (leitura)

- Exemplo - leitura linha a linha:

```
f = open('oMeuFicheiro.txt', 'r')
```

```
l1 = f.readline()
```

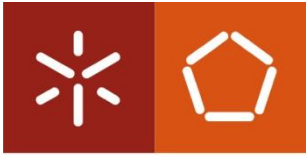
```
# lê uma linha do ficheiro
```

```
l2 = f.readline()
```

```
# lê a próxima linha do ficheiro
```

```
...
```

```
f.close()
```



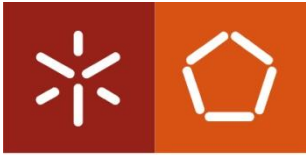
Python → Ficheiros de texto (leitura)

- Exemplo - leitura linhas para lista:

```
f = open('oMeuFicheiro.txt', 'r')
```

```
listaLinhas = f.readlines() # lê todas as linhas de um f para uma lista
```

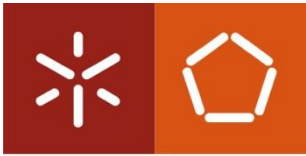
```
f.close()
```

Python → Ficheiros de texto (leitura)

- Exemplo (com ciclo): mais uma vez, o **for in** em python é útil e pode ser usado para facilmente para **ler o ficheiro linha a linha**. É, em muitos casos, a forma preferida quando podemos processar cada linha do ficheiro sem necessitar de guardar as linhas para uma lista.

```
f = open('oMeuFicheiro.txt', 'r')  
for linha in f: # Usar ciclo for para iterar as linhas do ficheiro  
    print(linha)    # dentro do ciclo, linha é uma string  
f.close()
```



Python → Ficheiros de texto (leitura)

- Exemplo com CSV
 - Usar o excel para criar um ficheiro csv.
 - Fazer um programa em Python para ler ficheiro e imprimir a linhas

#Exemplo ler um ficheiro excel gravado como .csv

```
f = open('pautaNotas.csv', 'r', encoding="utf-8")
```

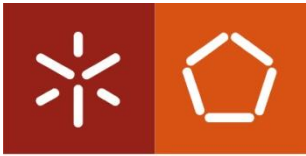
```
for l in f:
```

```
    lista = l.split(";") # separa a string por , e cria uma lista com os  
    elementos
```

```
    print(lista)
```

```
f.close()
```

pode ser necessário
dependo como o ficheiro
foi criado

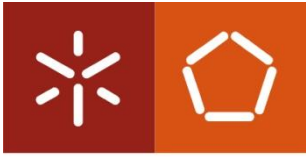


Python → Ficheiros de texto (leitura)

- Instrução delimitadora ***with***
 - em vez de:

```
file = open("filename", "r")  
file.read()  
file.close()
```
 - podemos simplesmente fazer:

```
with open("filename", "r") as file:  
    file.read()
```

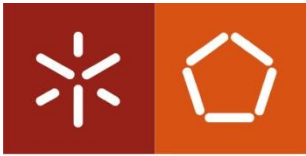


Python → Ficheiros de texto (leitura)

- Instrução delimitadora ***with***

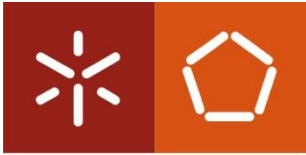
```
with open("filename", "r") as file:  
    file.read()
```

- Vantagem → no fim do bloco, não é necessário libertar o recurso com o ***close*** porque isso é feito automaticamente.



Python → Ficheiros de texto (escrita)

- Comandos para escrita:
 - `f.write(s)` # escreve **uma *string* s** para o ficheiro
 - `f.writelines(l)` # escreve uma **lista de *strings*** para o ficheiro



Ficheiros de Texto - Escrita

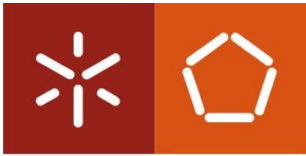
Universidade do Minho

- Exemplos – escrita de *string* simples:

```
f = open('oMeuFicheiro.txt', 'w')
```

```
f.write("Hello World!\n");
```

```
f.close()
```



Python → Ficheiros de texto (escrita)

- Exemplos – escrita de lista de *strings*:

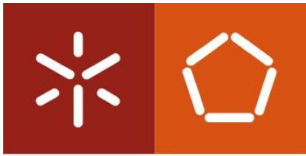
```
f = open('oMeuFicheiro.txt', 'w')
```

```
l = ["Uma linha de texto\n", "outra linha\n", "ainda outra"]
```

```
f.writelines(l)
```

#notar o `\n` no fim de cada string para escrever
em linhas separadas

```
f.close()
```



Python → Ficheiros de texto (escrita)

- Escrever um ficheiro csv com os valores da função seno entre 0 e 2π ; abrir o ficheiro em Excel e ver o gráfico.

```
import math
```

```
f = open('seno.csv', 'w')
```

```
angulos = [math.radians(i) for i in range(0,360)] # criar lista com os ângulos entre 0 e  $2\pi$ 
```

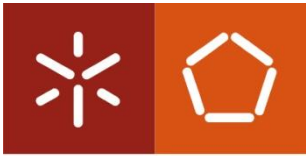
```
senos = [str(math.sin(i)) for i in angulos] # cria uma lista com seno para cada elemento da
```

```
# lista anterior usar o str para converter cada
```

```
# valor para string
```

```
f.write( ",".join(senos) )
```

```
f.close()
```

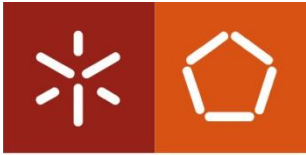
Universidade do Minho

Python → Ficheiros de texto (escrita)

- É, ainda, possível escrever num ficheiro csv usando **with**

`with open('seno.csv', 'w') as f:`

`...`



Universidade do Minho

Agora, vamos aos exercícios...



OBRIGADO PELA ATENÇÃO!